TP1

Victor Woelffel, Antoine Heitzmann, Dieudonné Lissom

December 2024

Table des matières

1 RISC-V Instruction Set **2**

2 RISC-V Tool Chain **7**

3 RISC-V Architecture **12**
3.1 Program Flow 12
3.2 Pipeline Diagram 14

4 Processor Design **16**
4.1 Instruction Set Architecture 16
4.2 Pipelining 23

5 Annexe **25**
5.1 Annexe RISC-V Instruction Set 25
5.2 Annexe RISC-V Tool Chain 27
5.3 Annexe RISC-V Architecture 31
5.4 Annexe Processor Design 33

1 RISC-V Instruction Set

Tout d'abord, la première étape consiste en la conversion des valeurs hexadécimales du programme fourni en représentation binaire. On en profite pour noter à chaque fois le format de l'instruction.

Adresse	Valeur Hexadécimale	Représentation Binaire	Format
0x0	00050893	00000000 00000101 00001000 10010011	I
0x4	00068513	00000000 00000110 10000101 00010011	I
0x8	04088063	00000100 00001000 10000000 01100011	SB
0xc	04058263	00000100 00000101 10000010 01100011	SB
0x10	04060063	00000100 00000110 00000000 01100011	SB
0x14	04d05063	00000100 11010000 01010000 01100011	SB
0x18	00088793	00000000 00001000 10000111 10010011	I
0x1c	00269713	00000000 00100110 10010111 00010011	I
0x20	00e888b3	00000000 11101000 10001000 10110011	R
0x24	0007a703	00000000 00000111 10100111 00000011	I
0x28	0005a803	00000000 00000101 10101000 00000011	I
0x2c	01070733	00000001 00000111 00000111 00110011	R
0x30	00e62023	00000000 11100110 00100000 00100011	S
0x34	00478793	00000000 01000111 10000111 10010011	I
0x38	00458593	00000000 01000101 10000101 10010011	I
0x3c	00460613	00000000 01000110 00000110 00010011	I
0x40	ff1792e3	11111111 00010111 10010010 11100011	SB
0x44	00008067	00000000 00000000 10000000 01100111	I
0x48	fff00513	11111111 11110000 00000101 00010011	I
0x4c	00008067	00000000 00000000 10000000 01100111	I
0x50	fff00513	11111111 11110000 00000101 00010011	I
0x54	00008067	00000000 00000000 10000000 01100111	I

TABLE 1 – Tableau des adresses, valeurs hexadécimales et représentations binaires.

On écrit ensuite un programme python qui permet de mieux comprendre les instructions du programme. Ce programme est donné en annexe.

La première partie du programme est composée de trois fonctions : "core_instruction_format()", "print_with_prefix" et "instruction_mnemonic".

— core_instruction_format() :
Cette fonction permet, à partir d'une variable globale PROGRAM qui regroupe et les instructions du programme fournit, et le format que l'on a déduit dans le tableau précédent, de retourner ce même programme mais selon le format d'instruction coeur (core instruction format) décrit dans la documentation à la fin du sujet.
Pour ce faire, il faut distinguer les cas en fonction des formats et simplement respecter les espaces de la documentation. On itère donc sur les bits d'une instruction en insérant les espaces aux bons endroits (on aura prix soin de précédemment retirer les espaces des instructions).

— print_with_prefix() :
Cette fonction a pour unique but d'afficher dans la console les instructions du programme sous la même forme que le programme initial, avec d'éventuels changements suite à l'exécution d'autres fonctions. Des variantes ont été utilisées pour notamment faire les tableaux latex de ce rapport.

— instruction_mnemonic() :
Cette fonction va nous aider à déterminer pour chaque instruction le mnémonique correspondant. Pour ce faire, le programme affiche le opcode et le funct3 selon la même disposition que dans la documentation et attend une réponse sur le mnémonique associé. Après avoir rentré le mnémonique, celui-ci est stocké un dictionnaire. L'ensemble des mnémonique sera affiché à la fin du programme et on pourra alors copier les valeurs pour les mettre dans une variable du programme et ainsi ne pas repartir de zéro à chaque exécution (nous vérifions effectivement que le mnémonique de chaque

instruction est bien vide).
Nous pouvons par ailleurs noter que cette fonction a été adaptée à la fonction "instruction_split()" décrite ci-dessous. Elle fonctionnait dans un premier temps avec une liste simple et les mots clefs "funct3" et "mnemonic", alors non définis, étaient simplement remplacés par des indices de liste pour chaque instruction.

Finalement, grâce au programme, on a réussi à mettre les instructions sous le format “core instruction”. Parallèlement, il nous aide plus facilement à déterminer les mnémoniques de chaque instruction. On obtient ainsi les instructions, toujours en binaires, mais cette fois avec leur mnémonic et avec leur découpe naturelle.

Adresse	**Instruction Binaire**	**Mnemonic**
0x0	000000000000 01010 000 10001 0010011	addi
0x4	000000000000 01101 000 01010 0010011	addi
0x8	0000010 00000 10001 000 00000 1100011	beq
0xc	0000010 00000 01011 000 00100 1100011	beq
0x10	0000010 00000 01100 000 00000 1100011	beq
0x14	0000010 01101 00000 101 00000 1100011	bge
0x18	000000000000 10001 000 01111 0010011	addi
0x1c	000000000010 01101 001 01110 0010011	slli
0x20	0000000 01110 10001 000 10001 0110011	add
0x24	000000000000 01111 010 01110 0000011	lw
0x28	000000000000 01011 010 10000 0000011	lw
0x2c	0000000 10000 01110 000 01110 0110011	add
0x30	0000000 01110 01100 010 00000 0100011	sw
0x34	000000000100 01111 000 01111 0010011	addi
0x38	000000000100 01011 000 01011 0010011	addi
0x3c	000000000100 01100 000 01100 0010011	addi
0x40	1111111 10001 01111 001 00101 1100011	bne
0x44	000000000000 00001 000 00000 1100111	jalr
0x48	111111111111 00000 000 01010 0010011	addi
0x4c	000000000000 00001 000 00000 1100111	jalr
0x50	111111111111 00000 000 01010 0010011	addi
0x54	000000000000 00001 000 00000 1100111	jalr

TABLE 2 – Tableau des instructions en binaire et leurs mnemonics correspondants.

On en déduit enfin, à l’aide des fonctions "instruction_split()" et "print_assembly()", le programme décodé en assembleur.

— print_assembly() :
La fonction porte bien son nom, elle permet d’afficher dans la console chaque instruction en assembleur. Pour ce faire, il est nécessaire de distinguer les formats de chaque instruction et de respecter les ordres des différentes parties que composent les instructions.

— instruction_split() :
Cette fonction a pour but de prendre le programme au format "core instruction", et pour chaque instruction classée dans une liste, de récupérer ses différents éléments, et de les stocker dans un dictionnaire. Ainsi, il sera plus facile d’y accéder, notamment pour l’affichage.
Ainsi, nous utilisons la fonction native split sur les instructions découpées selon le format instruction coeur. Ensuite, il suffit de distinguer les formats pour séléctionner les bons éléments au bon endroit. Il est important de noter que les éléments sont généralement convertis en décimal (par exemple les numéros des registres) mais les immédiats, eux, sont convertis en héxadécimal (et il faut bien penser à respecter la forme de ces derniers car ils sont la majorité du temps éparpillés dans l’instruction, tout est détaillé dans la documentation fournie dans le sujet).

Ainsi, cette partie du programme nous a permi de complètement déchiffrer les instruction et de les afficher en assembleur.

Pour rappel, les valeurs immédiates sont affichées en héxadécimal. Les registres gardent dans la colonne de gauche leur notation numérique tandis que les registres de la colonne de droite sont écrits selon leur notation symbolique ABI.

Adresse	Instructions notation xN	Instructions notation ABI
0x0	`addi, x17, x10, 0x0`	`addi, a7, a0, 0x0`
0x4	`addi, x10, x13, 0x0`	`addi, a0, a3, 0x0`
0x8	`beq, x17, x0, 0x40`	`beq, a7, zero, 0x40`
0xc	`beq, x11, x0, 0x44`	`beq, a1, zero, 0x44`
0x10	`beq, x12, x0, 0x40`	`beq, a2, zero, 0x40`
0x14	`bge, x0, x13, 0x40`	`bge, zero, a3, 0x40`
0x18	`addi, x15, x17, 0x0`	`addi, a5, a7, 0x0`
0x1c	`slli, x14, x13, 0x2`	`slli, a4, a3, 0x2`
0x20	`add, x17, x17, x14`	`add, a7, a7, a4`
0x24	`lw, x14, 0(x15)`	`lw, a4, 0(a5)`
0x28	`lw, x16, 0(x11)`	`lw, a6, 0(a1)`
0x2c	`add, x14, x14, x16`	`add, a4, a4, a6`
0x30	`sw, x14, 0(x12)`	`sw, a4, 0(a2)`
0x34	`addi, x15, x15, 0x4`	`addi, a5, a5, 0x4`
0x38	`addi, x11, x11, 0x4`	`addi, a1, a1, 0x4`
0x3c	`addi, x12, x12, 0x4`	`addi, a2, a2, 0x4`
0x40	`bne, x15, x17, 0x1fe4`	`bne, a5, a7, -(0x1C)`
0x44	`jalr, x0, x1, 0x0`	`jalr, zero, ra, 0x0`
0x48	`addi, x10, x0, 0xfff`	`addi, a0, zero, -1`
0x4c	`jalr, x0, x1, 0x0`	`jalr, zero, ra, 0x0`
0x50	`addi, x10, x0, 0xfff`	`addi, a0, zero, -1`
0x54	`jalr, x0, x1, 0x0`	`jalr, zero, ra, 0x0`

TABLE 3 – Comparaison des instructions avec registres en notation numérique xN et symbolique ABI.

Branch Delay Slots :

Les branch delay slots sont une technique utilisée dans les premières architectures de processeurs pipelinisés, notamment MIPS, pour gérer les risques de branchement. Ces risques surviennent lorsque le processeur ne sait pas immédiatement si un saut conditionnel sera pris ou non, ce qui peut perturber l'exécution fluide des instructions dans le pipeline.

Un branch delay slot est une instruction placée juste après une instruction de branchement. Elle est exécutée systématiquement, indépendamment du résultat du branchement. Cela permet d'exploiter efficacement le pipeline au lieu de le laisser inactif pendant que le processeur résout le branchement. Les compilateurs sont chargés de remplir ce slot avec une instruction utile, lorsque cela est possible.

Avantages :
— Optimisation des performances : En exécutant l'instruction dans le slot de délai, on évite des cycles d'inactivité (stalls) dans le pipeline, améliorant l'efficacité globale.
— Simplicité matérielle : Cette approche simplifie le matériel en évitant l'utilisation de mécanismes complexes comme la prédiction de branchement.

Inconvénients :
— Difficulté pour les compilateurs : Trouver une instruction adaptée pour remplir le slot est complexe. Si aucune instruction adéquate n'est disponible, un NOP (No Operation) est inséré, ce qui gaspille l'opportunité d'optimisation.
— Compatibilité limitée : Le code utilisant des slots de délai est moins portable, car ces optimisations dépendent fortement de l'architecture matérielle spécifique.

— Débogage compliqué : Les interruptions ou les points d'arrêt dans un slot de délai sont difficiles à gérer, car l'instruction dans le slot s'exécute toujours.

Finalement, les branch delay slots ont été une solution ingénieuse dans les premières architectures, mais ils ont été supplantés par des techniques plus avancées, mieux adaptées aux processeurs modernes.

Sources : UAF Computer Science, The Quick Advisor, Wikipedia

Pour pouvoir faire la suite, il est nécessaire de déterminer ce que font les instructions conditionnelles.

Branch Instruction	Source	Offset	Target	Target Instruction
`beq, a7, zero, 0x40`	`0x8`	`0x40`	`0x48`	`addi, a0, zero, -1`
`beq, a1, zero, 0x44`	`0xc`	`0x44`	`0x50`	`addi, a0, zero, -1`
`beq, a2, zero, 0x40`	`0x10`	`0x40`	`0x50`	`addi, a0, zero, -1`
`bge, zero, a3, 0x40`	`0x14`	`0x40`	`0x54`	`jalr, zero, ra, 0x0`
`bne, a5, a7, -(0x1c)`	`0x40`	`-(0x1c) (-28)`	`0x24`	`lw, a4, 0(a5)`

Table 4 – Résumé des branches conditionnelles et des cibles correspondantes.

On comprend alors que les 4 premières branches conditionnelles vérifient que la fonction peut bel et bien s'exécuter (dans le cas contraire cela mène à la fin du programme), tandis que la dernière branche conditionnelle forme une boucle. Cela nous permet d'obtenir un découpage global du programme :

Adresse	Instruction	Commentaire
Initialisation		
0x0	`addi, a7, a0, 0x0`	$a7 = a0$, paramètre n°1
0x4	`addi, a0, a3, 0x0`	$a0 = a3$, nombre de valeurs à copier
Tests conditionnels		
0x8	`beq, a7, zero, 0x40`	Si $a7 == 0$, saute à 0x48
0xc	`beq, a1, zero, 0x44`	Si $a1 == 0$, saute à 0x50
0x10	`beq, a2, zero, 0x40`	Si $a2 == 0$, saute à 0x50
0x14	`bge, zero, a3, 0x40`	Si $a3 \leq 0$, saute à 0x54
Calcul de fin d'adresse		
0x18	`addi, a5, a7, 0x0`	$a5 = a7$, début tableau src1
0x1c	`slli, a4, a3, 0x2`	$a4 = a3 * 4$, plage d'adresse à couvrir
0x20	`add, a7, a7, a4`	$a7 = a7 + a4$, calcul addresse limite
Boucle principale		
0x24	`lw, a4, 0(a5)`	$a4 = M[a5]$
0x28	`lw, a6, 0(a1)`	$a6 = M[a1]$
0x2c	`add, a4, a4, a6`	$a4 = a4 + a6$
0x30	`sw, a4, 0x0(a2)`	$M[a2] = a4$
0x34	`addi, a5, a5, 0x4`	$a5 = a5 + 4$, pointeur tableau src1
0x38	`addi, a1, a1, 0x4`	$a1 = a1 + 4$, pointeur tableau src2
0x3c	`addi, a2, a2, 0x4`	$a2 = a2 + 4$, pointeur tableau dest
0x40	`bne, a5, a7, 0x2fc`	Si $a5 \neq a7$, saute à 0x24
Fin de la fonction		
0x44	`jalr, zero, ra, 0x0`	succès de la fonction, retourne a0 (ok)
0x48	`addi, a0, zero, -1`	$a0 = -1$
0x4c	`jalr, zero, ra, 0x0`	retourne a0 (ko)
0x50	`addi, a0, zero, -1`	$a0 = -1$
0x54	`jalr, zero, ra, 0x0`	retourne a0 (ko)

Table 5 – Instructions et commentaires associés.

Finalement, on comprend l'utilité de la fonction présentée. On remarque que l'on a 3 tableaux en entrée : 2 tableaux sources (src1 et src2) et un tableau destination (dest).

Dans la "Boucle principale", les valeurs du tableau src1 sont sommées avec les valeurs du tableau scr2 tandis que le résultat est stocké dans le tableau dest.

La fin de la boucle est determinée par la partie "Calcul de fin d'adresse" : une adresse limite est calculée selon un nombre de valeurs fourni en entrée.

Finalement, le programme effectue une somme terme à terme de 2 tableaux en stockant le résultat dans un troisième tableau et avec un nombre de valeurs sommées déterminée en entrée.

Plusieurs valeurs peuvent être retournées par le programme :
- -1 si les 3 premiers tests conditionnels ne sont pas vérifiés.
- le nombre de termes à additionner fourni en paramètre dans le cas général, et cela même si l'addition n'a pas eu lieu (en cas d'un nombre de terme à additionner négatif, l'addition ne se fait pas mais c'est quand même la valeur qui est retournée). En effet, l'instruction à l'adresse 0x4 va fixer a0 qui ne sera pas modifiée sauf en cas d'entrée dans les 3 premiers tests conditionnels.

2 RISC-V Tool Chain

Après avoir analysé le programme pour en comprendre le fonctionnement, il est facile de créer un programme en C qui additionne un nombre fixe d'éléments issus de deux listes, tout en veillant à effectuer toutes les vérifications nécessaires.

```c
int addition_tableaux(int *src1, int *src2, int *dest, int values_nb) {

    int *a7 = src1;  // a7 = a0
    int a0 = values_nb;  // a0 = a3

    // Conditions
    if (a7 == 0) {
        return -1;  // Si a7 == 0, retour -1
    } if (src2 == 0) {
        return -1;  // Si a1 == 0, retour -1
    } if (dest == 0) {
        return -1;  // Si a2 == 0, retour -1
    } if (values_nb <= 0) {
        return a0;  // Si values_nb < 0, retourne 0
    }


    int value_range = values_nb;  // a4 = a3 * 4  (on ne multiplie pas par 4 car
    //l'objectif est d'avancer de 4 octets or a4 est un pointeur)
    a7 += value_range;  // a7 += a4

    // Boucle principale
    while (src1 != a7) {
        int src1_value = *src1;  // a4 = M[a5]
        int src2_value = *src2;  // a6 = M[a1]

        int somme = src1_value + src2_value;  // a4 = a4 + a6
        *dest = somme;  // M[a2] = a4

        // Mise à jour des pointeurs
        src1 += 1;  // a5 = a5 + 4
        src2 += 1;  // a1 = a1 + 4
        dest += 1;  // a2 = a2 + 4
    }
    return a0;  // Retour succès
}
```

Analyse du code désassemblé RISC-V

Vous trouverez en annexe les résultats de `riscv64-linux-gnu-objdump -d se201-prog_oX.o` pour les différentes compilations (x = "0" ou "" ou "3").

Comparaison du code de base et du code optimisé (-O0)

L'initialisation correspond aux instructions 0 à 18, tandis que le premier test conditionnel s'effectue juste après cette phase. Le second test est situé dans `L2`, le troisième dans `L4` et le dernier dans `L5`. La section `L6` est chargée d'effectuer le calcul de l'adresse limite. Quant à `L7`, elle représente la condition du *while* et détermine si l'on doit rester dans la boucle ou en sortir. La boucle principale se trouve dans `L8`, où les opérations d'addition et de mise à jour des pointeurs sont réalisées. Enfin, la fin de la fonction est marquée par `L3`.

On remarque que les instructions assembleur ne sont pas du tout les mêmes et que ce programme requiert beaucoup plus d'instructions pour réaliser chaque étape.

On note plusieurs différences notables :

1. **Gestion de la pile et des registres**
   Le compilateur génère un environnement sécurisé pour l'exécution de la fonction en respectant les conventions d'appel (ABI RISC-V) :

```
0:   fc010113           add     sp,sp,-64
4:   02812e23           sw      s0,60(sp)
8:   04010413           add     s0,sp,64
```

2. **Optimisations désactivées**
   Lorsque l'on compile avec l'option -O0, le compilateur désactive les optimisations pour faciliter le débogage. Le code est plus long car il effectue beaucoup de sauvegardes inutiles.

   Comparaison avec l'assembleur manuel
   Dans notre assembleur de base, les registres sont directement utilisés pour stocker les valeurs intermédiaires. Cela réduit les accès à la mémoire et minimise les instructions. Par exemple, au lieu de sauvegarder et recharger les valeurs depuis la pile, nous utilisons uniquement les registres.

Conclusion
La compilation avec -O0 génère un code assembleur beaucoup plus détaillé et verbeux que l'assembleur de base. Cela s'explique par l'absence d'optimisations et la rigueur du compilateur pour respecter les conventions d'appel (ABI RISC-V). Le compilateur produit un code générique, en utilisant davantag la mémoire pour faciliter le débogage. Cette compilation est réservée à une utilisation de test pour déboguer les programmes.

Contrairement à l'optimisation -O0, l'optimisation -O, possède énormément de similarités avec le programme de base

Comparaison du code de base et du code optimisé (-O)

1. Initialisation
Code de base :

```
0x0:  addi  a7, a0, 0          # a7 = a0 (src1)
0x4:  addi  a0, a3, 0          # a0 = a3 (values_nb)
```

Code optimisé (-O) :

```
0x0:  mv    a5, a0             # a5 = a0 (src1)
0x4:  mv    a0, a3             # a0 = a3 (values_nb)
```

Comparaison : Fonctionnellement identique, mais le code optimisé utilise `mv` pour plus de clarté.

2. Tests conditionnels
Code de base :

```
0x8:  beq   a7, zero, 0x40     # Si src1 == 0, saut à 0x48
0xc:  beq   a1, zero, 0x44     # Si src2 == 0, saut à 0x50
0x10: beq   a2, zero, 0x40     # Si dest == 0, saut à 0x50
0x14: bge   a3, zero, 0x40     # Si values_nb >= 0, saut à 0x54
```

Code optimisé (-O) :

```
0x8:  beqz  a5, 48 <.L4>       # Si src1 == 0, saut à 0x48
0xc:  beqz  a1, 50 <.L5>       # Si src2 == 0, saut à 0x50
0x10: beqz  a2, 58 <.L6>       # Si dest == 0, saut à 0x58
0x14: blez  a3, 5c <.L2>       # Si values_nb <= 0, saut à 0x5c
```

Comparaison : Le compilateur utilise `beqz` et `blez` au lieu de `beq` ou `bge` avec `zero`, simplifiant le code en limitant les accès à la mémoire.

3. Calcul de l'adresse limite
Code de base :

```
0x18: addi   a5, a7, 0x0
0x1c: slli   a4, a3, 0x2          # a4 = values_nb * 4
0x20: add    a7, a7, a4           # a7 = src1 + (values_nb * 4)
```

Code optimisé (-O) :

```
0x18: sll    a7, a3, 0x2          # a7 = values_nb * 2
0x1c: add    a7, a5, a7           # a7 = src1 + (values_nb * 16)
```

Comparaison : l'instruction `0x20: beq a5, a7, 5c <.L2>` n'est pas dans le programme de base. Cette instruction dit que si a7 est nulle (car a7=a7+a5) alors la fonction renvoie la valeur de retour. cette fonction représente le cas où src1 = a7 dans le while..

4. Boucle principale

Code de base :

```
0x24: lw     a4, 0(a5)            # Charger src1
0x28: lw     a6, 0(a1)            # Charger src2
0x2c: add    a4, a4, a6           # Addition
0x30: sw     a4, 0(a2)            # Stocker dest
0x34: addi   a5, a5, 0x4          # Avancer src1
0x38: addi   a1, a1, 0x4          # Avancer src2
0x3c: addi   a2, a2, 0x4          # Avancer dest
0x40: bne    a5, a7, 0x2fc        # Reboucler
```

Code optimisé (-O) :

```
0x24: lw     a4, 0(a5)            # Charger src1
0x28: lw     a6, 0(a1)            # Charger src2
0x2c: add    a4, a4, a6           # Addition
0x30: sw     a4, 0(a2)            # Stocker dest
0x34: add    a5, a5, 4            # Avancer src1
0x38: add    a1, a1, 4            # Avancer src2
0x3c: add    a2, a2, 4            # Avancer dest
0x40: bne    a7, a5, 24 <.L3>     # Reboucler
```

Comparaison : Même logique, en remplaçant les addi par des add, qui est juste une modificaiton de l'affichage effectué par objdump.

5. Fin de la fonction

Code de base :

```
0x44: jalr   zero, ra, 0x0        # Retour normal
0x48: addi   a0, zero, -1         # Retour -1
0x4c: jalr   zero, ra, 0x0
0x50: addi   a0, zero, -1         # Retour -1
0x54: jalr   zero, ra, 0x0
```

Code optimisé (-O) :

```
0x44: ret                         # Retour normal
0x48: li     a0, -1               # Charger -1
0x4c: ret
0x50: li     a0, -1
0x54: ret
0x58: li     a0, -1
0x5c: ret
```

Comparaison : Le code optimisé utilise `ret` et `li` pour simplifier les retours et chargements de constantes, notons que dans ce code il y a 3 fois le retour avec la valeur -1 et que les 3 sont utilisé (une pour chaque comparaison avec 0 "beqz").

En conclusion, le code optimisé est plus clair et exécute moins d'instructions inutiles, se rapprochant davantage du code d'assemblage manuel en exploitant mieux les registres et les instructions simplifiées. Il est aussi plus simple à lire.

Comparaison du code de base et du code optimisé (-O3)

Voici une comparaison détaillée entre le code de base (fourni en annexe) et le code optimisé obtenu avec l'option -O3. Nous identifions les similitudes, les différences et les améliorations apportées par le compilateur.

1. Initialisation

Code de base :

```
0x0:  addi  a7, a0, 0          # a7 = a0 (src1)
0x4:  addi  a0, a3, 0          # a0 = a3 (values_nb)
```

Code optimisé (-O3) :

```
0x0:  mv    a5, a0             # a5 = a0 (src1)
0x40: mv    a0, a3             # a0 = a3 (values_nb)
```

Comparaison : Le code optimisé utilise `mv` pour simplifier les copies de valeurs, évitant l'usage inutile de `addi`. et `mv` est utilisé plus tard (ligne 0x40) car pas utile avant.

2. Tests conditionnels

Code de base :

```
0x8:  beq   a7, zero, 0x40     # Si src1 == 0, saut à 0x48
0xc:  beq   a1, zero, 0x44     # Si src2 == 0, saut à 0x50
0x10: beq   a2, zero, 0x40     # Si dest == 0, saut à 0x50
0x14: bge   a3, zero, 0x40      # Si values_nb >= 0, saut à 0x54
```

Code optimisé (-O3) :

```
0x4:  beqz  a0, 48 <.L8>       # Si src1 == 0, saut à erreur
0x8:  beqz  a1, 48 <.L8>       # Si src2 == 0, saut à erreur
0xc:  beqz  a2, 48 <.L8>       # Si dest == 0, saut à erreur
0x10: blez  a3, 40 <.L4>       # Si values_nb <= 0, saut à fin
```

Comparaison : de la même manière que pour la compilation -o, l'assembleur utilise beqz plutôt que beq pour limiter les accès mémoire. On remarque que toutes les instructions vont à la même adresse car ils renvoient la même chose.

3. Calcul de l'adresse limite

Code de base :

```
0x18: addi  a5, a7, 0x0        # a5 = src1
0x1c: slli  a4, a3, 0x2        # a4 = values_nb * 4
0x20: add   a7, a7, a4         # a7 = src1 + (values_nb * 4)
```

Code optimisé (-O3) :

```
0x14: sll   a7, a3, 0x2        # a7 = values_nb * 4
0x18: add   a7, a0, a7         # a7 = src1 + (values_nb * 16)
0x1c: beq   a0,a7,40 <.L4>     # entrée de le while (ou non)
```

Comparaison : Exactement comme l'optimisation -o, le code optimisé calcule directement avec un décalage de 4 bits (x16) plutôt que 2 (x4)..

4. Boucle principale

Code de base :

```
0x24: lw    a4, 0(a5)          # Charger src1
0x28: lw    a6, 0(a1)          # Charger src2
0x2c: add   a4, a4, a6         # Addition
0x30: sw    a4, 0x0(a2)        # Stocker dest
0x34: addi  a5, a5, 0x4        # Avancer src1
0x38: addi  a1, a1, 0x4        # Avancer src2
0x3c: addi  a2, a2, 0x4        # Avancer dest
0x40: bne   a5, a7, 0x2fc      # Reboucler
```

Code optimisé (-O3) :

```
0x20: lw     a4, 0(a5)           # Charger src1
0x24: lw     a6, 0(a1)           # Charger src2
0x28: add    a5, a5, 4           # Avancer src1
0x2c: add    a1, a1, 4           # Avancer src2
0x30: add    a4, a4, a6          # Addition
0x34: sw     a4, 0(a2)           # Stocker dest
0x38: add    a2, a2, 4           # Avancer dest
0x3c: bne    a7, a5, 20 <.L5>    # Reboucler
```

Comparaison : Même logique, mais le code optimisé est plus compact, utilisant `add` pour avancer les pointeurs.

5. Fin de la fonction

Code de base :

```
0x44: jalr   zero, ra, 0x0       # Retour normal
0x48: addi   a0, zero, -1        # Retour -1
0x4c: jalr   zero, ra, 0x0
```

Code optimisé (-O3) :

```
0x40: mv     a0, a3              # a0 = values_nb
0x44: ret                        # Retour normal
0x48: li     a0, -1              # a0 = -1
0x4c: ret
```

Comparaison : Le compilateur utilise `ret` pour simplifier le retour et `li` pour charger directement la constante `-1`.

Conclusion

Le code optimisé obtenu avec -O3 est légèrement plus compact que le code de base et n'accède au registre que lorsqu'il veut les utiliser. Il utilise des instructions simplifiées comme `mv`, `ret`, `beqz` et `li` évitant ainsi les redondances (notons qu'il n'y a aucun `addi`). Cette optimisation est très proche de celle `-o` et donc proche du programme du code assembleur.

3 RISC-V Architecture

3.1 Program Flow

Dans cette partie, nous analysons les différents incidents non prévus pouvant survenir dans un pipeline RISC-V.

1. Problèmes liés aux données (Data Hazard)

Lorsqu'une instruction dépend des résultats d'une instruction, mais que ces résultats ne sont pas encore disponibles cela pose un problème.

par exemple :

```
lw a1, 0(a0)        // Charger une donnée dans a1
add a2, a1, a3      // Utiliser la donnée de a1
```

Dans ce cas, l'instruction `add` doit attendre que l'instruction `lw` termine son instruction pour pouvoir accéder à la donnée de a1 (qui sera chargée en mémoire à l'étape WB).

2. Problèmes liés aux branchements (Control Hazard)

Lorsqu'il y a une incertitude sur le flux d'exécution du programme, généralement causée par des instructions de branchement (`beq`, `bne`, `jal`) où l'on peut rencontrer des imprévus.

par exemple :

```
beq a1, a2, Label    // Brancher si a1 == a2
add a3, a3, a4       // Instruction potentiellement inutile
sub a5, a5, a6       // Instruction potentiellement inutile
```

On ne sait si la branche est prise qu'à la fin de l'étape Execute (EX). Or le processeur peut continuer lire les instructions suivantes, ce qui peut poser problème.

3. Problèmes d'instructions concurencielles (Structural Hazard)

Par moment, plusieurs instructions concurrentes nécessitent la même ressource matérielle, comme une mémoire ou une unité d'exécution.

par exemple :

```
lw a1, 0(a0) // Accéder à la mémoire
sw a2, 4(a0) // Accéder à la mémoire
```

Ici, `lw` et `sw` tentent d'accéder à la mémoire pendant le même cycle ce qui n'est pas possible.

Ces problèmes peuvent être gérés à l'aide de différentes méthodes (stalling, forwarding, flushing), nous expliquerons comment dans la partie Pipeline Diagram.

PC	Instruction				a0	a1	a2	a3	a4	a5	a6	a7	Explication		Adresse	Valeur
0x0	**addi**	a7	a0	0	0x200	0x200	0x200	0x2	0x0	0x0	0x0	**0x200**	copie de la valeur de a0 dans a7		Initialisation	
0x4	**addi**	a0	a3	0	**0x2**	0x200	0x200	0x2	0x0	0x0	0x0	0x200	prépare la valeur de retour		0x200	0x61
0x8	**beq**	a7	zero	0x40	0x2	0x200	0x200	0x2	0x0	0x0	0x0	0x200	vérifie que a7!=0		0x204	0x20
0xc	**beq**	a1	zero	0x44	0x2	0x200	0x200	0x2	0x0	0x0	0x0	0x200	vérifie que a1!=0		0x208	0x62
0x10	**beq**	a2	zero	0x40	0x2	0x200	0x200	0x2	0x0	0x0	0x0	0x200	vérifie que a2!=0		0x20c	0x0
0x14	**bge**	zero	a3	0x40	0x2	0x200	0x200	0x2	0x0	0x0	0x0	0x200	vérifie que a3>=0		0x210	0x0
0x18	**addi**	a5	a7	0	0x2	0x200	0x200	0x2	0x0	**0x200**	0x0	0x200	a5 devient l'adresse du début du tableau src1			
0x1c	**slli**	a4	a3	0x2	0x2	0x200	0x200	0x2	**0x8**	0x200	0x0	0x200	calcul la longueure de la plage d'adresse à couvrir			
0x20	**add**	a7	a7	a4	0x2	0x200	0x200	0x2	0x8	0x200	0x0	**0x208**	calcul l'adresse limite			
0x24	**lw**	a4	0(a5)		0x2	0x200	0x200	0x2	**0x61**	0x200	0x0	0x208	a4 devient la valeur du 1er élément de la liste 1			
0x28	**lw**	a6	0(a1)		0x2	0x200	0x200	0x2	0x61	0x200	**0x61**	0x208	a6 devient la valeur du 1er élément de la liste 2			
0x2c	**add**	a4	a4	a6	0x2	0x200	0x200	0x2	**0xc2**	0x200	0x61	0x208	a4 devient la somme des deux éléments des listes			
0x30	**sw**	a4	0(a2)		0x2	0x200	0x200	0x2	0xc2	0x200	0x61	0x208	modifie la valeur à l'adresse 0x200	----->	0x200	0xc2
0x34	**addi**	a5	a5	0x4	0x2	0x200	0x200	0x2	0xc2	**0x204**	0x61	0x208	incrément l'adresse de lecture de src1		0x204	0x20
0x38	**addi**	a1	a1	0x4	0x2	**0x204**	0x200	0x2	0xc2	0x204	0x61	0x208	incrément l'adresse de lecture de src2		0x208	0x62
0x3c	**addi**	a2	a2	0x4	0x2	0x204	**0x204**	0x2	0xc2	0x204	0x61	0x208	incrément l'adresse de écriture de la list de sortie		0x20c	0x0
0x40	**bne**	a5	a7	0x2fc	0x2	0x204	0x204	0x2	0xc2	0x204	0x61	0x208	retour dans la boucle car a5!=a7		0x210	0x0
0x24	**lw**	a4	0(a5)		0x2	0x204	0x204	0x2	**0x20**	0x204	0x61	0x208	a4 devient la valeur du 2ème élément de la liste 1			
0x28	**lw**	a6	0(a1)		0x2	0x204	0x204	0x2	0x20	0x204	**0x20**	0x208	a6 devient la valeur du 2ème élément de la liste 2			
0x2c	**add**	a4	a4	a6	0x2	0x204	0x204	0x2	**0x40**	0x204	0x20	0x208	a4 devient la somme des deux éléments des listes			
0x30	**sw**	a4	0(a2)		0x2	0x204	0x204	0x2	0x40	0x204	0x20	0x208	modifie la valeur à l'adresse 0x204	----->	0x200	0xc2
0x34	**addi**	a5	a5	0x4	0x2	0x204	0x204	0x2	0x40	**0x208**	0x20	0x208	incrément l'adresse de lecture de src1		0x204	0x40
0x38	**addi**	a1	a1	0x4	0x2	**0x208**	0x204	0x2	0x40	0x208	0x20	0x208	incrément l'adresse de lecture de src2		0x208	0x62
0x3c	**addi**	a2	a2	0x4	0x2	0x208	**0x208**	0x2	0x40	0x208	0x20	0x208	incrément l'adresse de écriture de la list de sortie		0x20c	0x0
0x40	**bne**	a5	a7	0x2fc	0x2	0x208	0x208	0x2	0x40	0x208	0x20	0x208	on sort de la boucle car a5==a7		0x210	0x0
0x44	**jalr**	zero	ra	0x0	0x2	0x208	0x208	0x2	0x40	0x208	0x20	0x208	fin du programme			

3.2 Pipeline Diagram

Dans cette deuxième partie, nous allons créer un diagramme de pipeline.

Pour cela, il est crucial de bien comprendre les notions de stalling, forwarding et flushing. Nous les introduisons rapidement ci-dessous, mais ils sont décrits précisément dans le cours.

— **Stalling** : Le stalling se produit lorsqu'un processeur retarde l'exécution d'une instruction pour résoudre les conflits de données (dépendances), créant ainsi des cycles d'attente afin que les données nécessaires soient disponibles. Le stalling permettrait en théorie de résoudre l'ensemble des risques rencontrés lors de la partie précédente. cependant, ce n'est pas très efficace.
— **Forwarding** : Le forwarding est une technique qui permet de transmettre directement des données entre les étapes intermédiaires du pipeline pour éviter les délais dus aux conflits de données (et donc du stalling), permettant ainsi d'accéder immédiatement aux données nécessaires ce qui nous permettra dans certains cas d'éviter le stalling.
— **Flushing** : Le flushing consiste à annuler les instructions dans le pipeline après une instruction de branchement, notamment pour résoudre les risques liés aux branchements, afin d'éviter l'exécution d'instructions basées sur des prédictions incorrectes.

Finalement, nous avons cerné les concepts cruciaux du pipeline. Ainsi, nous utiliserons l'échelle suivante pour décrire le diagramme présenté page suivante (ainsi qu'en annexe), tout en mettant en évidence ces concepts clefs :

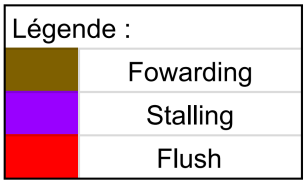


FIGURE 1 – Légende du diagramme de pipeline.

PC	Instruction				Clock	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35
0x0	addi	a7	a0	0		IF	ID	EX	MEM	WB																																
0x4	addi	a0	a3	0			IF	ID	EX	MEM	WB																															
0x8	beq	a7	zero	0x40				IF	ID	**EX**	MEM	WB																														
0xc	beq	a1	zero	0x44					IF	ID	EX	MEM	WB																													
0x10	beq	a2	zero	0x40						IF	ID	EX	MEM	WB																											
0x14	bge	zero	a3	0x40							IF	ID	EX	MEM	WB																										
0x18	addi	a5	a7	0								IF	ID	EX	MEM	WB																								
0x1c	slli	a4	a3	0x2									IF	ID	EX	MEM	WB																							
0x20	add	a7	a7	a4										IF	ID	**EX**	MEM	WB																						
0x24	lw	a4	0(a5)												IF	ID	EX	MEM	WB																					
0x28	lw	a6	0(a1)													IF	ID	EX	MEM	WB																				
0x2c	add	a4	a4	a6													IF	ID	**ID**	**ID**	EX	MEM	WB																	
0x30	sw	a4	0(a2)															IF	**IF**	**IF**	**ID**	EX	MEM	WB																
0x34	addi	a5	a5	0x4																	IF	ID	EX	MEM	WB															
0x38	addi	a1	a1	0x4																		IF	ID	EX	MEM	WB														
0x3c	addi	a2	a2	0x4																			IF	ID	EX	MEM	WB													
0x40	bne	a5	a7	0x2fc																				IF	ID	EX	MEM	WB												
0x44	jalr	zero	ra	0x0																					IF	ID	**flush**	**flush**	**flush**											
0x48	addi	a0	zero	-1																						IF	**flush**	**flush**	**flush**	**flush**										
0x24	lw	a4	0(a5)																								IF	ID	EX	MEM	WB									
0x28	lw	a6	0(a1)																									IF	ID	EX	MEM	WB								
0x2c	add	a4	a4	a6																									IF	ID	**ID**	**ID**	EX	MEM	WB					
0x30	sw	a4	0(a2)																											IF	**IF**	IF	ID	EX	MEM	WB				
0x34	addi	a5	a5	0x4																												IF	ID	EX	MEM	WB				
0x38	addi	a1	a1	0x4																													IF	ID	EX	MEM	WB			
0x3c	addi	a2	a2	0x4																														IF	ID	EX	MEM	WB		
0x40	bne	a5	a7	0x2fc																															IF	ID	EX	MEM	WB	
0x44	jalr	zero	ra	0x0																																IF	ID	EX	MEM	WB

4 Processor Design

4.1 Instruction Set Architecture

Notre objectif dans cette partie sera de décrire les instructions d'un processeur avec des contraintes spécifiques.

Description des 16 registres.

Le tableau suivant décrit la fonction 16 registres :

Registre	Nom	Usage
r15	`pc`	Compteur de programme.
r14	`lr`	Adresse de retour (*Link register*).
r13	`sp`	Pointeur de pile (*Stack Pointer*).
r12	`ip`	Registre pour appels intra-procédure (*Intra-Procedure Call*).
r11	`fp`/v7	Pointeur de frame (ou registre variable 7).
r10	`v6`	Registre variable 6 (*callee-saved*).
r9	`v5`	Registre variable 5 (*callee-saved*).
r8	`v4`	Registre variable 4 (*callee-saved*).
r7	`v3`	Registre variable 3 (*callee-saved*).
r6	`v2`	Registre variable 2 (*callee-saved*).
r5	`v1`	Registre variable 1 (*callee-saved*).
r4	`a3`	Argument 4 ou registre temporaire (*caller-saved*).
r3	`a2`	Argument 3 ou registre temporaire (*caller-saved*).
r2	`a1`	Argument 2 ou registre temporaire/résultat (*caller-saved*).
r1	`a0`	Argument 1 ou registre temporaire/résultat (*caller-saved*).
r0	`zero`	Valeur fixée à zéro.

TABLE 6 – Utilisation des registres du processeur

Passage des arguments lors des appels de fonction

Le passage des arguments dans un appel de fonction dépend du nombre d'arguments, si il y a entre 1 et 4 arguments, les quatre premiers arguments sont passés directement dans les registres `a0` à `a3`. Si il y a plus de 4 arguments, Les arguments supplémentaires sont placés dans la pile (*stack*), à partir de l'adresse pointée par le pointeur de pile (`sp`).

Cette convention permet un accès rapide aux arguments les plus courants grâce à l'utilisation des registres, tout en laissant la pile gérer les cas plus complexes.

Retour d'un appel de fonction et récupération des résultats

Les résultats d'une fonction sont stockés dans le registre `a0`. Si une fonction retourne plusieurs valeurs, les registres `a1`, `a2`, et `a3` peuvent être utilisés pour contenir les valeurs supplémentaires. De plus, lorsqu'une fonction termine son exécution, elle utilise l'adresse stockée dans le registre `lr` (Link Register) pour revenir à l'instruction située juste après l'appel de fonction.

Conventions de préservation et modification des registres

Lors d'un appel de fonction, certains registres doivent être préservés par la fonction appelée, tandis que d'autres peuvent être modifiés. Les registres `r4` à `r10` sont considérés comme des variables sauvegardées *callee-saved*. Ils doivent être restaurés dans leur état initial par la fonction appelée avant de retourner. Les registres `r0` à `r3` (utilisés pour les arguments et comme registres temporaires) ainsi que le registre `r12` (`ip`) peuvent être modifiés par la fonction appelée. Le programme appelant doit sauvegarder leur contenu si nécessaire. Il y a aussi des **Registres spéciaux :**

— `r13` (`sp`) : Le pointeur de pile peut être modifié mais doit être restauré à la fin de la fonction.
— `r14` (`lr`) : Le registre contenant l'adresse de retour est écrasé à chaque appel de fonction.
— `r15` (`pc`) : Le compteur de programme est toujours modifié au cours de l'exécution.

Description des différentes instructions

Pour pouvoir tout mettre sous forme binaire, on associe à chaque registre un numéro. Comme il y a 16 registres, il nous faut donc 4 bits par registre

Numéro	Registres
0000	pc
0001	lr
0010	sp
0011	ip
0100	fp/v7
0101	v6
0110	v5
0111	v4
1000	v3
1001	v2
1010	v1
1011	a3
1100	a2
1101	a1
1110	a0
1111	zero

TABLE 7 – Numérotation des registres

Nous allons désormais décrire toutes les instructions avec dans un premier temps l'ecriture en assembleur et ensuite l'écriture en binaire.
Nous devons définir les **instructions arithmétiques**, les **instructions Load and Store**, les instructions **MOVI**, **BNEZ** et **JUMP**

Instruction arithmétique

Les instructions ADD, SUB, AND, OR font une opération sur 2 registres (`t1` et `t2`) et les stockent dans un troisième registre.

Voici ce que fait chaque instruction
— `ADD r3, r1, r2 ; r3 = r1 + r2` *r3 est le registre stockant le résultat (peu importe le registre)*
— `SUB r3, r1, r2 ; r3 = r1 - r2` *r1 et r2 sont deux opérandes (peu importe les registres)*
— `AND r3, r1, r2 ; r3 = r1 & r2`
— `OR r3, r1, r2 ; r3 = r1 | r2`

Voici comment les instructions sont encodées :

INSTRUCTION	opcode (2 bits)	fct1 (1 bit)	fct2 (1 bit)	rs2 (4 bits)	rs1 (4 bits)	rd (4 bits)
ADD a0, t1, t2	00	0	0	0111	0110	1110
SUB a0, t1, t2	00	0	1	0111	0110	1110
AND a0, t1, t2	00	1	0	0111	0110	1110
OR a0, t1, t2	00	1	1	0111	0110	1110

TABLE 8 – Encodage des instructions arithmétiques

Instruction LOAD et STORE

Format Assembleur :
— `LOAD rx, IMM(ry)` ; Charge la valeur à l'adresse `[ry + IMM]` dans `rx`.
— `STORE rx, IMM(ry)` ; Stocke la valeur de `rx` à l'adresse `[ry + IMM]`.

INSTRUCTION	opcode (2 bits)	fct1 (1 bit)	IMM (5 bits)	rs (4 bits)	rd (4 bits)
LOAD t0, IMM(t1)	01	0	IMM	0110	0101
STORE t0, IMM(t1)	01	1	IMM	0101	0110

Table 9 – Encodage des instructions LOAD et STORE

Instruction immédiate (MOVI)

Format assembleur :
`MOVI t0, IMM` ; `t0 = IMM` où `t0` est un registre et `IMM` un immédiat.

Voici comment elle est encodée :

INSTRUCTION	opcode (2 bits)	fct1 (1 bit)	IMM (9 bits)	rd
MOVI t0, IMM	10	0	IMM	0101

Table 10 – Encodage de l'instruction MOVI

Instruction Branchement si non nul (BNEZ)

Format Assembleur :
`BNEZ rD, Imm` ; Si `rD` $\neq 0$, saut vers `PC + Imm` où `rD` est un registre quelconque.

Voici comment elle est encodée :

INSTRUCTION	opcode (2 bits)	IMM (10 bits)	rd (4 bits)
BNEZ t0, IMM	11	IMM	0101

Table 11 – Encodage de l'instruction BNEZ

Instruction de saut inconditionnel (JUMP)

Format assembleur :
`JUMP rd` ; `PC = rd` où `rd` est un registre quelconque.

Voici comment elle est encodée :

INSTRUCTION	opcode (2 bits)	fct1 (1 bit)	fct2 (1 bit)	reserved (8 bits)	rd (4 bits)
JUMP t1	10	1	0	00000000	0110

Table 12 – Encodage de l'instruction JUMP

Instruction d'appel de fonction (CALL)

Format assembleur :
`CALL Imm ;` Appelle la fonction à l'adresse `PC + Imm`.

Voici comment elle est encodée :

INSTRUCTION	OPCODE (2 bits)	fct1 (1 bit)	fct2 (1 bit)	Imm (12 bits)
CALL Imm	10	1	1	Imm

Table 13 – Encodage de l'instruction CALL

Groupe d'Instruction

Pour rendre la doc plus lisible, il est intéressant de regrouper les instructions dans des formats en fonction de l'organisation de leurs encodage binaire

— **R-format** : Regroupera toutes les instructions arithémtiques.

— **S-format** : ce format regroupera les instructions manipulant la mémoire.
— **I-format** : Utilisé pour les instructions qui permettent de stoquer des immuables dans des registres
— **SB-format** : Utilisé pour les instructions de branchement conditionnel.
— **J-format** : Utilisé pour les instructions de saut.
— **C-format** : Utilisé pour les instructions de saut avec rappel de fonction.

Instruction	Format
ADD	R
SUB	R
AND	R
OR	R
LOAD	S
STORE	S
MOVI	I
BNEZ	SB
JUMP	J
CALL	C

Table 14 – Association des instructions et leurs formats

Instruction NOP

Pour pouvoir coder plus facilement, il peut être intéressant de créer une instruction NOP.
L'instruction NOP (No-Operation) est une pseudo-instruction qui ne fait rien, ne modifie aucun registre ni mémoire. Elle est utile pour aligner des instructions, remplir des slots de délai de branchement, et pour le débogage.
L'instruction NOP ne requiert pas d'opcode unique, car on peut la créer grâce à des opérations déjà existantes.
La pseudo-instruction est traduite par :

```
ADD r0, r0, r0 ; r0 = r0 + r0 (ne fait rien puisque r0 est toujours 0)
```

Encodage de l'instruction NOP

INSTRUCTION	OPCODE (2 bits)	fct2 (1 bit)	fct1 (1 bit)	RS2 (4 bits)	RS1 (4 bits)
NOP	00	0	0	0000	0000

Table 15 – Encodage de l'instruction NOP

Traduction du code de la question 1

Après avoir créé toutes nos instructions, on nous demande de traduire en assembleur, compatible avec notre processeur, le code de la question 1. L'objectif est de s'assurer que notre processeur fonctionne correctement et de comprendre comment les différentes instructions sont codées.

Nous rappelons le code que l'on doit implémenter :

Adresse	Instructions notation xN	Instructions notation ABI
0x0	`addi, x17, x10, 0x0`	`addi, a7, a0, 0x0`
0x4	`addi, x10, x13, 0x0`	`addi, a0, a3, 0x0`
0x8	`beq, x17, x0, 0x40`	`beq, a7, zero, 0x40`
0xc	`beq, x11, x0, 0x44`	`beq, a1, zero, 0x44`
0x10	`beq, x12, x0, 0x40`	`beq, a2, zero, 0x40`
0x14	`bge, x0, x13, 0x40`	`bge, zero, a3, 0x40`
0x18	`addi, x15, x17, 0x0`	`addi, a5, a7, 0x0`
0x1c	`slli, x14, x13, 0x2`	`slli, a4, a3, 0x2`
0x20	`add, x17, x17, x14`	`add, a7, a7, a4`
0x24	`lw, x14, 0(x15)`	`lw, a4, 0(a5)`
0x28	`lw, x16, 0(x11)`	`lw, a6, 0(a1)`
0x2c	`add, x14, x14, x16`	`add, a4, a4, a6`
0x30	`sw, x14, 0(x12)`	`sw, a4, 0(a2)`
0x34	`addi, x15, x15, 0x4`	`addi, a5, a5, 0x4`
0x38	`addi, x11, x11, 0x4`	`addi, a1, a1, 0x4`
0x3c	`addi, x12, x12, 0x4`	`addi, a2, a2, 0x4`
0x40	`bne, x15, x17, 0x1fe4`	`bne, a5, a7, -(0x1c)`
0x44	`jalr, x0, x1, 0x0`	`jalr, zero, ra, 0x0`
0x48	`addi, x10, x0, 0xfff`	`addi, a0, zero, -1`
0x4c	`jalr, x0, x1, 0x0`	`jalr, zero, ra, 0x0`
0x50	`addi, x10, x0, 0xfff`	`addi, a0, zero, -1`
0x54	`jalr, x0, x1, 0x0`	`jalr, zero, ra, 0x0`

TABLE 16 – Programme en assembleur de la question 1

Avec nos instructions, cela donne le résultat suivant :

```
0x0  ADD v1, a0, zero;  v1 <=> a7
0x2  ADD a0, a3, zero;
0x4  BNEZ v1, 0x4;
0x6  JUMP 0x3c;
0x8  BNEZ a1, 0x4;
0xa  JUMP 0x40;
0xc  BNEZ a2, 0x4;
0xe  JUMP 0x36;
0x10 MOVI v2, 0b100000000;      bge avec 0 <=> bit poids ford = 1
0x12 MOVI v3, #23;              v2 devient 0b11...1100000000 (32 bits)
0x14 MOVI v4, #1;
0x16 SUB v3, v3, v4;
0x18 ADD v2, v2, v2;            multiplié par 2 <=> SLL (uniquement ici)
0x1a BNEZ v3, 0b111111100;      revient de 2 instruction en arrière
0x1c AND v2,v2,a3;
0x1e BNEZ v2, 0x2e;             v2,..,v4 à nouveau libre
0x20 ADD v2, v1, zero;          v2 <=> a5
0x22 ADD v3, a3,a3;             slli de 2 <=> multiplication par 4
0x24 ADD v3, v3,v3;             v3 <=> a4
0x26 ADD v1, v1, v3;
0x28 LOAD v1, 0(v2);
0x2a LOAD v4, 0(a1);            v4 <=> a6
0x2c ADD v1, v1, v4;
0x2e STORE v1, 0(a2);           v3 à nouveau libre
0x30 MOVI v3, #4;               v3 à pour valeur 4
0x32 ADD v2, v2, v3;
0x34 ADD a1, a1, v3;
0x36 ADD a2, a2, v3;
0x38 SUB v5, v2, v1;            bne <=> SUB puis BNEZ
0x3a BNEZ v5, 0b1111110100;     retour de 0x12, v1 libéré
0x3c ADD v1, lr, zero;          JALR <=> lr=0 + branchement
0x3e ADD lr, zero, zero;
0x40 JUMP v1;
0x42 MOVI a0, -1;
0x44 ADD v1, lr, zero;          l'instrucion jalr puis addi y est 2 fois
0x46 ADD lr, zero, zero;
0x48 JUMP v1;
0x4a MOVI a0, -1;
0x4c ADD v1, lr, zero;
0x4e ADD lr, zero, zero;
0x50 JUMP v1;
```

Nous allons expliquer par quoi nous avons remplacé les instructions qui ne sont pas évidentes.

bge

La première difficulté que l'on rencontre concerne l'implémentation de BGE, qui signifie "plus grand ou égal". Notre stratégie est de soustraire le deuxième élément au premier. Après avoir fait cela, il ne nous reste plus qu'à analyser le signe. Cependant, nous ne disposons pas d'une instruction qui le permet directement. Pour cela, nous effectuons un AND entre notre résultat et 1000...0 (comportant 8 zeros car l'immédiat fait 9 bits) afin de ne conserver que le bit de poids fort. Cela nous donne les instructions suivantes :

```
0x10 MOVI v2, 0b100000000;      bge avec 0 <=> bit poids ford = 1
0x12 MOVI v3, #23;              v2 devient 0b11...1100000000 (32 bits)
0x14 MOVI v4, #1;
0x16 SUB v3, v3, v4;
0x18 ADD v2, v2, v2;            multiplié par 2 <=> SLL (uniquement ici)
0x1a BNEZ v3, 0b111111100;      revient de 2 instruction en arrière
0x1c AND v2,v2,a3;
0x1e BNEZ v2, 0x2e;
```

slli

Cette instruction décale tous les bits vers la gauche (ce qui équivaut ici à une multiplication par 4, car la valeur est positive d'après l'instruction bge). On peut donc simplement multiplier par 4 en exécutant ces instructions :

```
0x22 ADD v3, a3,a3;             slli de 2 <=> multiplication par 4
0x24 ADD v3, v3,v3;
```

jalr

L'instruction JALR (Jump and Link Register) effectue un saut vers une adresse calculée en ajoutant une valeur immédiate à un registre source, tout en sauvegardant l'adresse de retour dans un registre de destination. On peut donc remplacer l'instruction par ces instructions :

```
0x44 ADD v1, lr, zero;
0x46 ADD lr, zero, zero;
0x48 JUMP v1;
```

4.2 Pipelining

Design du processeur

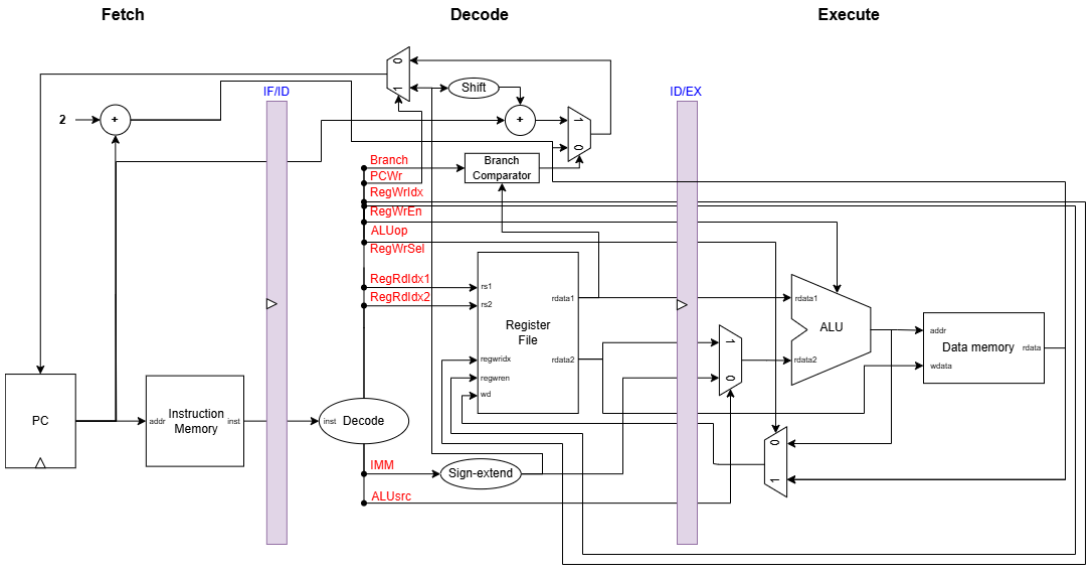


FIGURE 2 – Design du processeur

La structure global du processeur ne change pas beaucoup. Les changements principaux concernent la gestion du jump et du call (développé ci-dessous), la gestion de la mémoire qui n'est plus dans la partie MEM. En effet, la mémoire doit être gérée durant la partie d'exécution (EX), ce qui implique d'utiliser le résultat de l'ALU directement, ainsi qu'éventuellement le résultat du register file en cas de STORE d'un registre. De plus, le PC + 2 de la partie Fetch (IF) est sortie de IF/ID car sinon il produit un décalage sur la pipeline dès la première instruction. Les signaux de contrôle ont bien évidemment été nouvellement reliés pour correspondre à notre design actuel.
Nous avons effectué chaque instruction définie dans la partie précédente sur le design présenté et il semblerait qu'il implémente effectivement chacune de nos instructions.

Instruction CALL :

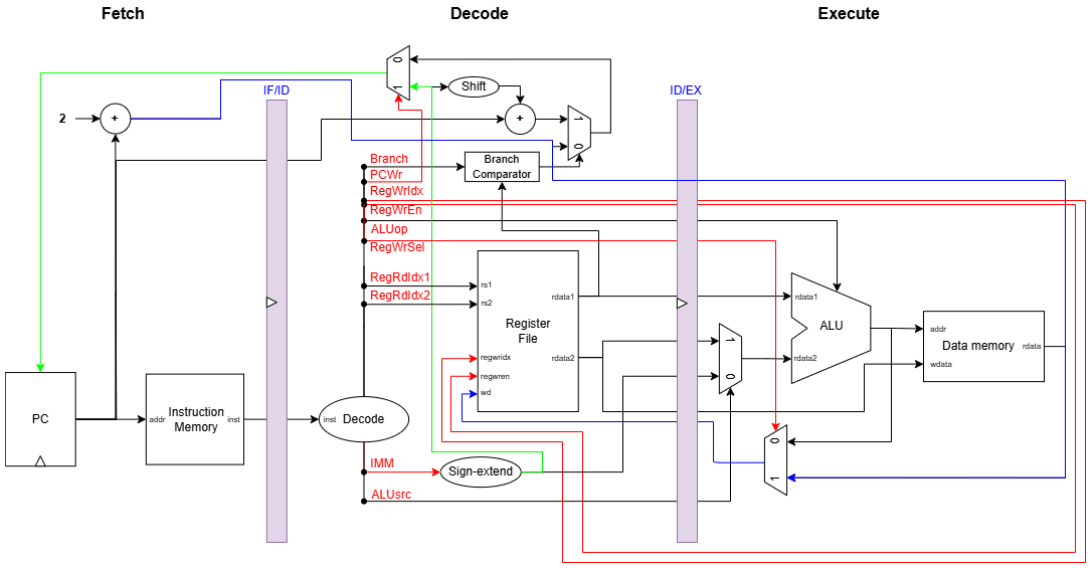


FIGURE 3 – Focus sur l'instruction CALL

Légende de la figure 3 :
— Les lignes rouges sont les signaux de contrôle qui agissent durant l'exécution du CALL.
— Les lignes bleue contiennent l'adresse PC + 2, qui sera ainsi stockée dans le registre (lr).
— Les lignes vertes contiennent l'immédiat et par suite l'adresse de PC après le CALL.

Instruction Fetch (IF)
— Le Program Counter (PC) pointe l'adresse de l'instruction CALL.
— L'instruction CALL est ainsi récupérée depuis la mémoire d'instructions : elle est de la forme CALL IMM.

Instruction Decode (ID)
— L'instruction CALL est décodée pour identifier : Son type (dans ce cas, un appel inconditionnel avec opérande immédiat uniquement). Les informations nécessaires pour le traitement (immédiat spécifiant l'adresse de saut).
— L'immédiat est directement dirigé vers le multiplexeur contrôlé par le signal de contrôle noté "PCWr" sur le schéma. Il passe aussi par le bloc "sign-extend" mais l'immédiat n'étant pas relatif, cela ne change rien à la valeur.
— Le PC est alors directement mis à jour avec la nouvelle adresse, correspondant à l'immédiat décodé. L'instruction suivante aura donc lieu comme souhaitée.

Exécution (EX)
— La valeur du PC + 2 (puisque chaque instruction occupe 2 octets sur ce processeur 16 bits) est ensuite redirigée pour être écrite dans le registre de liaison r14 (lr), qui l'enregistre.
— Pour ce faire, le signal "RegWrSel" est actif pour transmettre la valeur au fichier des registres. De plus, "RegWrEn" est actif car on cherche à écrire dans un registre, et le numéro du registre en question, directement connu par le decode, est transmis par le signal "RegWrIdx".
— Les autres signaux générés lors du decode ne sont pas utilisés (notamment AluOP, AluSrc, RegRdIdx1/2...)

Résolution des dangers :

— **Dangers de données :** Il peut y en avoir, mais contrairement à un pipeline à 5 étages, nous pouvons ici utiliser systématiquement le forwarding en cas de besoin d'une donnée uniquement disponible dans la partie exécution (EX).
— **Dangers de contrôle :** Nous ne sommes pas concernés par ce danger : en effet, le PC est directement mis à jour en cas de branchement. Une des conséquence est que nous n'avons pas besoin de nous occuper des flush, comme expliqué ci-dessous
— **Dangers de structure :** Même si notre pipeline réduit les chances de ce danger, il n'élimine pas tous les risques, et les dangers de structure sont toujours possibles. Nous pourrions mettre en place des stalls pour régler ce problème.

Flushing :

La logique de suppression d'instructions (flush) est nécessaire dans le cas général pour éviter l'exécution d'instructions incorrectes après des sauts. Cependant, dans notre processeur, les branchements sont pris en compte dès le decode (ID). Ainsi, le PC est mis directement à jour et le fetch (IF) a lieu sur l'instruction voulue. Nous n'avons donc pas besoin du flush.

5 Annexe

5.1 Annexe RISC-V Instruction Set

format "core instruction"

```
def core_instruction_format():
    i = 0
    program = []
    for instruction, format in PROGRAM:
        instruction = instruction.replace(" ", "")
        instruction = instruction[::-1]

        if format in ["R", "S", "SB"]:
            for bit in range(len(instruction)):
                if bit in [7, 12+1, 15+2, 20+3, 25+4]:
                    instruction = instruction[:bit] + " " + instruction[bit:]
                    bit += 1
        elif format == "I":
            for bit in range(len(instruction)):
                if bit in [7, 12+1, 15+2, 20+3]:
                    instruction = instruction[:bit] + " " + instruction[bit:]
                    bit += 1
        elif format in ["U", "UJ"]:
            for bit in range(len(instruction)):
                if bit in [7, 12+1]:
                    instruction = instruction[:bit] + " " + instruction[bit:]
                    bit += 1

        instruction = instruction[::-1]
        program.append((instruction, format))
    return program

def print_with_prefix(program):
    for i in range(len(program)):
        print(hex(4*i) + ": " + str(program[i][0]))

def instruction_mnemonic(program, mnemonics):
    for i in range(len(program)):
        instruction = program[i]
        mnemonic = mnemonics[i]
        if mnemonic == "":
            if "funct3" not in instruction.keys():
                print("unimplemented instruction U or UJ")
            else:
                print(instruction["opcode"] + "    " + instruction["funct3"])
                result = input()
                instruction["mnemonic"] = result
        else:
            instruction["mnemonic"] = mnemonic
    print([instruction["mnemonic"] for instruction in program])
```

Transformation binaire, instruction assembleur

```
def print_assembly(program):
    for i in range(len(program)):
        instruction = program[i]
        if instruction["format"] == "R":
            print(hex(4*i) + ": " + instruction["mnemonic"] + ", x" +
                  instruction["rd"] + ", x" + instruction["rs1"] + ", x" +
                  instruction["rs2"])
        elif instruction["format"] == "I":
            print(hex(4*i) + ": " + instruction["mnemonic"] + ", x" +
                  instruction["rd"] + ", x" + instruction["rs1"] + ", " +
                  instruction["imm"])
        elif instruction["format"] == "S":
            print(hex(4*i) + ": " + instruction["mnemonic"] + ", x" +
                  instruction["rs2"] + ", " + instruction["imm"] +
                  "(x" + instruction["rs1"] + ")")
        elif instruction["format"] == "SB":
            print(hex(4*i) + ": " + instruction["mnemonic"] + ", x" +
                  instruction["rs1"] + ", x" + instruction["rs2"] + ", " +
                  instruction["imm"])
        else:
            print("unimplemented format")


def instruction_split(program, mnemonics):
    program2 = []
    for instruction, mnemonic in zip(program, mnemonics):
        instruction, format = instruction
        instruction = instruction.split(" ")
        if format == "R":
            funct7, rs2, rs1, funct3, rd, opcode = instruction
            program2.append({"funct7":funct7, "rs2":str(int(rs2, 2)),
                             "rs1":str(int(rs1, 2)), "funct3":funct3,
                             "rd":str(int(rd, 2)), "opcode":opcode,
                             "format":format, "mnemonic":mnemonic})
        elif format == "I":
            imm11_0, rs1, funct3, rd, opcode = instruction
            program2.append({"imm":str(hex(int(imm11_0, 2))),
                            "rs1":str(int(rs1, 2)), "funct3":funct3,
                            "rd":str(int(rd, 2)), "opcode":opcode,
                             "format":format, "mnemonic":mnemonic})
        elif format == "S":
            imm11_5, rs2, rs1, funct3, imm4_0, opcode = instruction
            imm = str(hex(int(imm11_5+imm4_0, 2)))
            program2.append({"imm":imm, "rs2":str(int(rs2, 2)),
                             "rs1":str(int(rs1, 2)), "funct3":funct3,
                             "opcode":opcode, "format":format,
                             "mnemonic":mnemonic})
        elif format == "SB":
            imm10_5, rs2, rs1, funct3, imm4_1, opcode = instruction
            imm = str(hex(int(imm10_5[0]+imm4_1[4]+imm10_5[1:]+imm4_1[:4]
                              +'0', 2)))
            program2.append({"imm":imm, "rs2":str(int(rs2, 2)),
                             "rs1":str(int(rs1, 2)), "funct3":funct3,
                             "opcode":opcode, "format":format,
                             "mnemonic":mnemonic})
        else:
            print("error, unsupported core instruction format")
            return -1
    return program2
```

5.2 Annexe RISC-V Tool Chain

Compilation pour -O0, Déassemblage de la section .text :

```
se201-prog_2_o0.o:     format de fichier elf32-littleriscv

00000000 <addition_tableaux>:
   0:   fc010113                add     sp,sp,-64
   4:   02812e23                sw      s0,60(sp)
   8:   04010413                add     s0,sp,64
   c:   fca42623                sw      a0,-52(s0)
  10:   fcb42423                sw      a1,-56(s0)
  14:   fcc42223                sw      a2,-60(s0)
  18:   fcd42023                sw      a3,-64(s0)
  1c:   fcc42783                lw      a5,-52(s0)
  20:   fef42623                sw      a5,-20(s0)
  24:   fc042783                lw      a5,-64(s0)
  28:   fef42423                sw      a5,-24(s0)
  2c:   fec42783                lw      a5,-20(s0)
  30:   00079663                bnez    a5,3c <.L2>
  34:   fff00793                li      a5,-1
  38:   0bc0006f                j       f4 <.L3>

0000003c <.L2>:
  3c:   fc842783                lw      a5,-56(s0)
  40:   00079663                bnez    a5,4c <.L4>
  44:   fff00793                li      a5,-1
  48:   0ac0006f                j       f4 <.L3>

0000004c <.L4>:
  4c:   fc442783                lw      a5,-60(s0)
  50:   00079663                bnez    a5,5c <.L5>
  54:   fff00793                li      a5,-1
  58:   09c0006f                j       f4 <.L3>

0000005c <.L5>:
  5c:   fc042783                lw      a5,-64(s0)
  60:   00f04663                bgtz    a5,6c <.L6>
  64:   fe842783                lw      a5,-24(s0)
  68:   08c0006f                j       f4 <.L3>

0000006c <.L6>:
  6c:   fc042783                lw      a5,-64(s0)
  70:   fef42223                sw      a5,-28(s0)
  74:   fe442783                lw      a5,-28(s0)
  78:   00279793                sll     a5,a5,0x2
  7c:   fec42703                lw      a4,-20(s0)
  80:   00f707b3                add     a5,a4,a5
  84:   fef42623                sw      a5,-20(s0)
  88:   05c0006f                j       e4 <.L7>

0000008c <.L8>:
  8c:   fcc42783                lw      a5,-52(s0)
  90:   0007a783                lw      a5,0(a5)
  94:   fef42023                sw      a5,-32(s0)
  98:   fc842783                lw      a5,-56(s0)
  9c:   0007a783                lw      a5,0(a5)
  a0:   fcf42e23                sw      a5,-36(s0)
  a4:   fe042703                lw      a4,-32(s0)
```

```
  a8:   fdc42783                lw      a5,-36(s0)
  ac:   00f707b3                add     a5,a4,a5
  b0:   fcf42c23                sw      a5,-40(s0)
  b4:   fc442783                lw      a5,-60(s0)
  b8:   fd842703                lw      a4,-40(s0)
  bc:   00e7a023                sw      a4,0(a5)
  c0:   fcc42783                lw      a5,-52(s0)
  c4:   00478793                add     a5,a5,4
  c8:   fcf42623                sw      a5,-52(s0)
  cc:   fc842783                lw      a5,-56(s0)
  d0:   00478793                add     a5,a5,4
  d4:   fcf42423                sw      a5,-56(s0)
  d8:   fc442783                lw      a5,-60(s0)
  dc:   00478793                add     a5,a5,4
  e0:   fcf42223                sw      a5,-60(s0)

000000e4 <.L7>:
  e4:   fcc42703                lw      a4,-52(s0)
  e8:   fec42783                lw      a5,-20(s0)
  ec:   faf710e3                bne     a4,a5,8c <.L8>
  f0:   fe842783                lw      a5,-24(s0)

000000f4 <.L3>:
  f4:   00078513                mv      a0,a5
  f8:   03c12403                lw      s0,60(sp)
  fc:   04010113                add     sp,sp,64
 100:   00008067                ret
```

Compilation pour -O, Déassemblage de la section .text :

```

se201-prog_2_o.o:     format de fichier elf32-littleriscv

00000000 <addition_tableaux>:
   0:   00050793                mv      a5,a0
   4:   00068513                mv      a0,a3

00000008 <.LVL1>:
   8:   04078063                beqz    a5,48 <.L4>
   c:   04058263                beqz    a1,50 <.L5>
  10:   04060463                beqz    a2,58 <.L6>
  14:   04d05463                blez    a3,5c <.L2>

00000018 <.LVL2>:
  18:   00269893                sll     a7,a3,0x2
  1c:   011788b3                add     a7,a5,a7

00000020 <.LVL3>:
  20:   03178e63                beq     a5,a7,5c <.L2>

00000024 <.L3>:
  24:   0007a703                lw      a4,0(a5)
  28:   0005a803                lw      a6,0(a1)
  2c:   01070733                add     a4,a4,a6

00000030 <.LVL5>:
  30:   00e62023                sw      a4,0(a2)

00000034 <.LVL6>:
  34:   00478793                add     a5,a5,4
  38:   00458593                add     a1,a1,4

0000003c <.LVL8>:
  3c:   00460613                add     a2,a2,4

00000040 <.LBE2>:
  40:   fef892e3                bne     a7,a5,24 <.L3>
  44:   00008067                ret

00000048 <.L4>:
  48:   fff00513                li      a0,-1
  4c:   00008067                ret

00000050 <.L5>:
  50:   fff00513                li      a0,-1

00000054 <.LVL12>:
  54:   00008067                ret

00000058 <.L6>:
  58:   fff00513                li      a0,-1

0000005c <.L2>:
  5c:   00008067                ret
```

Compilation pour -O3, Déassemblage de la section .text :

```
se201-prog_2_o3.o:     format de fichier elf32-littleriscv

00000000 <addition_tableaux>:
   0:   00050793                mv      a5,a0

00000004 <.LVL1>:
   4:   04050263                beqz    a0,48 <.L8>
   8:   04058063                beqz    a1,48 <.L8>
   c:   02060e63                beqz    a2,48 <.L8>
  10:   02d05863                blez    a3,40 <.L4>

00000014 <.LVL2>:
  14:   00269893                sll     a7,a3,0x2
  18:   011508b3                add     a7,a0,a7

0000001c <.LVL3>:
  1c:   03150263                beq     a0,a7,40 <.L4>

00000020 <.L5>:
  20:   0007a703                lw      a4,0(a5)
  24:   0005a803                lw      a6,0(a1)
  28:   00478793                add     a5,a5,4

0000002c <.LVL5>:
  2c:   00458593                add     a1,a1,4

00000030 <.LVL6>:
  30:   01070733                add     a4,a4,a6

00000034 <.LVL7>:
  34:   00e62023                sw      a4,0(a2)

00000038 <.LVL8>:
  38:   00460613                add     a2,a2,4

0000003c <.LBE2>:
  3c:   fef892e3                bne     a7,a5,20 <.L5>

00000040 <.L4>:
  40:   00068513                mv      a0,a3
  44:   00008067                ret

00000048 <.L8>:
  48:   fff00513                li      a0,-1

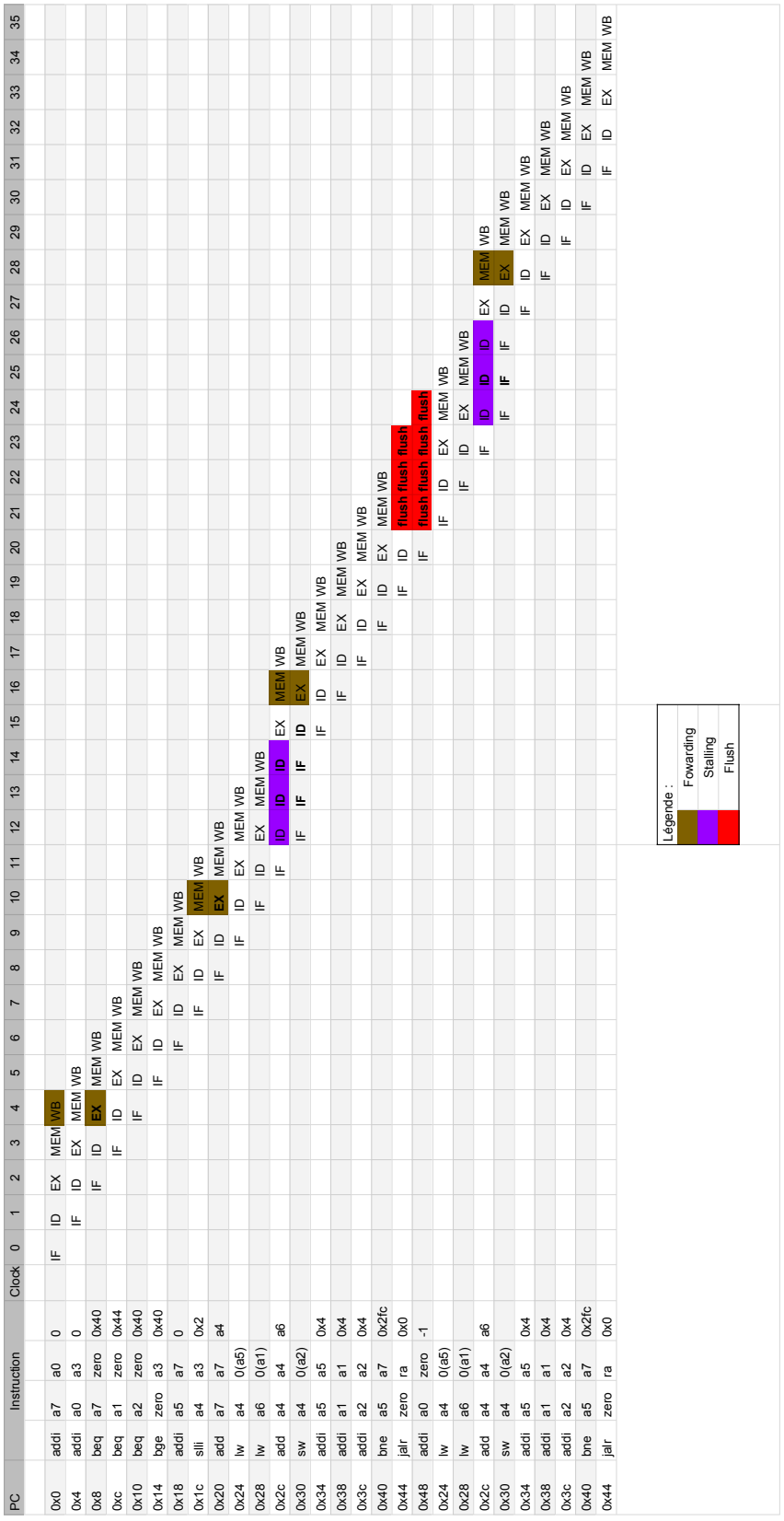
0000004c <.LVL12>:
  4c:   00008067                ret
```

5.3 Annexe RISC-V Architecture

Program Flow

PC	Instruction				a0	a1	a2	a3	a4	a5	a6	a7	Explication		Adresse	Valeur
0x0	**addi**	a7	a0	0	0x200	0x200	0x200	0x2	0x0	0x0	0x0	**0x200**	copie de la valeur de a0 dans a7		Initialisation	
0x4	**addi**	a0	a3	0	**0x2**	0x200	0x200	0x2	0x0	0x0	0x0	0x200	prépare la valeur de retour		0x200	0x61
0x8	**beq**	a7	zero	0x40	0x2	0x200	0x200	0x2	0x0	0x0	0x0	0x200	vérifie que a7!=0		0x204	0x20
0xc	**beq**	a1	zero	0x44	0x2	0x200	0x200	0x2	0x0	0x0	0x0	0x200	vérifie que a1!=0		0x208	0x62
0x10	**beq**	a2	zero	0x40	0x2	0x200	0x200	0x2	0x0	0x0	0x0	0x200	vérifie que a2!=0		0x20c	0x0
0x14	**bge**	zero	a3	0x40	0x2	0x200	0x200	0x2	0x0	0x0	0x0	0x200	vérifie que a3>=0		0x210	0x0
0x18	**addi**	a5	a7	0	0x2	0x200	0x200	0x2	0x0	**0x200**	0x0	0x200	a5 devient l'adresse du début du tableau src1			
0x1c	**slli**	a4	a3	0x2	0x2	0x200	0x200	0x2	**0x8**	0x200	0x0	0x200	calcul la longueure de la plage d'adresse à couvrir			
0x20	**add**	a7	a7	a4	0x2	0x200	0x200	0x2	0x8	0x200	0x0	**0x208**	calcul l'adresse limite			
0x24	**lw**	a4	0(a5)		0x2	0x200	0x200	0x2	**0x61**	0x200	0x0	0x208	a4 devient la valeur du 1er élément de la liste 1			
0x28	**lw**	a6	0(a1)		0x2	0x200	0x200	0x2	0x61	0x200	**0x61**	0x208	a6 devient la valeur du 1er élément de la liste 2			
0x2c	**add**	a4	a4	a6	0x2	0x200	0x200	0x2	**0xc2**	0x200	0x61	0x208	a4 devient la somme des deux éléments des listes			
0x30	**sw**	a4	0(a2)		0x2	0x200	0x200	0x2	0xc2	0x200	0x61	0x208	modifie la valeur à l'adresse 0x200	----->	0x200	0xc2
0x34	**addi**	a5	a5	0x4	0x2	0x200	0x200	0x2	0xc2	**0x204**	0x61	0x208	incrément l'adresse de lecture de src1		0x204	0x20
0x38	**addi**	a1	a1	0x4	0x2	**0x204**	0x200	0x2	0xc2	0x204	0x61	0x208	incrément l'adresse de lecture de src2		0x208	0x62
0x3c	**addi**	a2	a2	0x4	0x2	0x204	**0x204**	0x2	0xc2	0x204	0x61	0x208	incrément l'adresse de écriture de la list de sortie		0x20c	0x0
0x40	**bne**	a5	a7	0x2fc	0x2	0x204	0x204	0x2	0xc2	0x204	0x61	0x208	retour dans la boucle car a5!=a7		0x210	0x0
0x24	**lw**	a4	0(a5)		0x2	0x204	0x204	0x2	**0x20**	0x204	0x61	0x208	a4 devient la valeur du 2ème élément de la liste 1			
0x28	**lw**	a6	0(a1)		0x2	0x204	0x204	0x2	0x20	0x204	**0x20**	0x208	a6 devient la valeur du 2ème élément de la liste 2			
0x2c	**add**	a4	a4	a6	0x2	0x204	0x204	0x2	**0x40**	0x204	0x20	0x208	a4 devient la somme des deux éléments des listes			
0x30	**sw**	a4	0(a2)		0x2	0x204	0x204	0x2	0x40	0x204	0x20	0x208	modifie la valeur à l'adresse 0x204	----->	0x200	0xc2
0x34	**addi**	a5	a5	0x4	0x2	0x204	0x204	0x2	0x40	**0x208**	0x20	0x208	incrément l'adresse de lecture de src1		0x204	0x40
0x38	**addi**	a1	a1	0x4	0x2	**0x208**	0x204	0x2	0x40	0x208	0x20	0x208	incrément l'adresse de lecture de src2		0x208	0x62
0x3c	**addi**	a2	a2	0x4	0x2	0x208	**0x208**	0x2	0x40	0x208	0x20	0x208	incrément l'adresse de écriture de la list de sortie		0x20c	0x0
0x40	**bne**	a5	a7	0x2fc	0x2	0x208	0x208	0x2	0x40	0x208	0x20	0x208	on sort de la boucle car a5==a7		0x210	0x0
0x44	**jalr**	zero	ra	0x0	0x2	0x208	0x208	0x2	0x40	0x208	0x20	0x208	fin du programme			

Pipeline Diagram



5.4 Annexe Processor Design

Programmme selon les instructions de notre processeur

```
0x0  ADD v1, a0, zero;  v1 <=> a7
0x2  ADD a0, a3, zero;
0x4  BNEZ v1, 0x4;
0x6  JUMP 0x3c;
0x8  BNEZ a1, 0x4;
0xa  JUMP 0x40;
0xc  BNEZ a2, 0x4;
0xe  JUMP 0x36;
0x10 MOVI v2, 0b100000000;      bge avec 0 <=> bit poids ford = 1
0x12 MOVI v3, #23;              v2 devient 0b11...1100000000 (32 bits)
0x14 MOVI v4, #1;
0x16 SUB v3, v3, v4;
0x18 ADD v2, v2, v2;            multiplié par 2 <=> SLL (uniquement ici)
0x1a BNEZ v3, 0b111111100;      revient de 2 instruction en arrière
0x1c AND v2,v2,a3;
0x1e BNEZ v2, 0x2e;             v2,..,v4 à nouveau libre
0x20 ADD v2, v1, zero;          v2 <=> a5
0x22 ADD v3, a3,a3;             slli de 2 <=> multiplication par 4
0x24 ADD v3, v3,v3;             v3 <=> a4
0x26 ADD v1, v1, v3;
0x28 LOAD v1, 0(v2);
0x2a LOAD v4, 0(a1);            v4 <=> a6
0x2c ADD v1, v1, v4;
0x2e STORE v1, 0(a2);           v3 à nouveau libre
0x30 MOVI v3, #4;               v3 à pour valeur 4
0x32 ADD v2, v2, v3;
0x34 ADD a1, a1, v3;
0x36 ADD a2, a2, v3;
0x38 SUB v5, v2, v1;            bne <=> SUB puis BNEZ
0x3a BNEZ v5, 0b1111110100;     retour de 0x12, v1 libéré
0x3c ADD v1, lr, zero;          JALR <=> lr=0 + branchement
0x3e ADD lr, zero, zero;
0x40 JUMP v1;
0x42 MOVI a0, -1;
0x44 ADD v1, lr, zero;          l'instrucion jalr puis addi y est 2 fois
0x46 ADD lr, zero, zero;
0x48 JUMP v1;
0x4a MOVI a0, -1;
0x4c ADD v1, lr, zero;
0x4e ADD lr, zero, zero;
0x50 JUMP v1;
```